

Repechage

Agentic Payment Recovery
for Failed Razorpay Checkouts

AI reasons. Policy decides. Execution is controlled.

Ayushi Singh
Razorpay AI Buildathon — Track 03

From failed payment to controlled recovery

01

The Problem

Why revenue disappears

02

The Solution

How Repechage thinks

03

The System

Architecture & safety

04

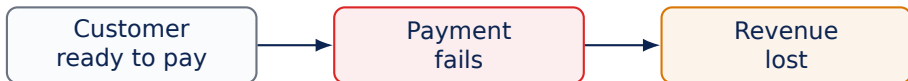
The Evidence

Baseline vs. AI

We will end with what actually worked — and what didn't.

The customer was ready to pay.

Then the payment failed.



The opportunity existed. The recovery decision didn't.

So why should they receive the same response?

card_declined

insufficient_funds

otp_timeout

network_error

Naive automation

Same response

The consequence

- Wasted recovery attempts
- Annoyed customers
- Missed recoverable payments
- No clear stopping point

Different failures need different decisions.

The opportunity is measurable.

₹98,24,112

total transaction value in our 1,000-event benchmark

66.6%

of revenue flagged at-risk

₹65,38,889

438

genuinely recoverable events

if the right action is taken quickly

This is not just a payment problem. It is a revenue recovery problem.

**It knows what happened.
It doesn't understand what to do next.**

1

No diagnosis

Failure reasons are treated similarly.

2

No stopping rule

Unrecoverable payments keep getting contacted.

3

No audit trail

Merchants cannot easily explain automated actions.

Detect. Diagnose. Recover.



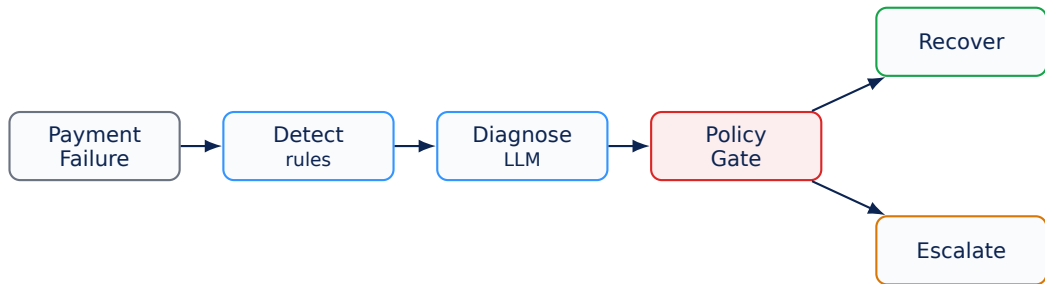
An agent that reasons about the recovery decision — not just the failure.

But the AI does not get the final say.

**AI reasons.
Policy decides.
Execution is controlled.**



The recovery workflow



Every decision produces an audit trail.

It cannot invent an action.

01	02	03	04	05
recover_now	send_payment_link	wait_and_retry	escalate_to_merchant	stop

Five actions. No free-form execution.

The AI recommends. Hard rules can still say NO.

Hard stops

- Attempts $\geq 3 \rightarrow$ **STOP**
- Already recovered $\rightarrow$ **STOP**
- Amount $> ₹5,000 \rightarrow$ **ESCALATE**

Execution limits

- Maximum 10 real actions per run
- Live execution defaults to **OFF**
- Invalid or low-confidence outputs are gated

When in doubt: do not execute. Escalate instead.

Deterministic Detector

status
failure_reason
previous_recovery_attempts



Risk classification

Explainable · Zero LLM cost

LLM Recovery Agent

Rich transaction context



Diagnosis + Action + Confidence

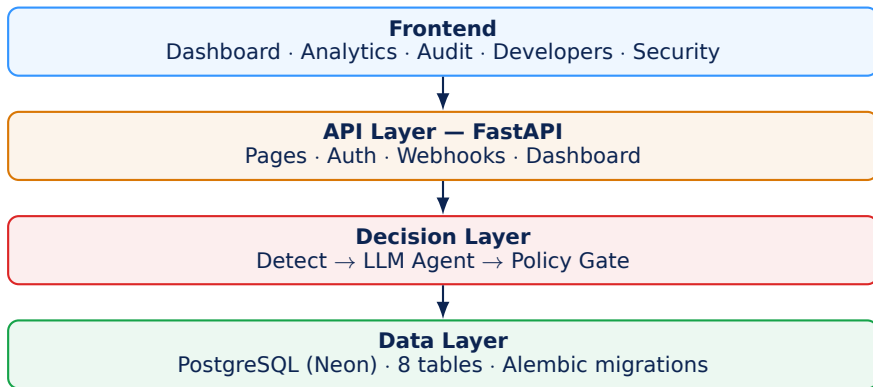
Contextual reasoning · Structured output

Neither model sees ground truth. Neither can bypass policy.

Not a paragraph. A structured decision.

```
{  
  "diagnosis": "..."  
  "recovery_probability": 0.82  
  "recommended_action": "send_payment_link"  
  "reason": "..."  
  "confidence": 0.91  
}
```

Schema validated → post-filtered → policy gated



The system is designed to be explainable after the fact.

Transaction lifecycle

- transactions
- synthetic_events
- detection_results
- agent_decisions

Execution lifecycle

- recovery_attempts
- audit_logs
- webhook_events
- merchants

What happened? Why? When? What action was taken?

The audit layer keeps the answer.

Better than what?



Both systems are evaluated on the exact same 662 at-risk events under the same simulation economics.

662

at-risk events evaluated

Deterministic Baseline

662 candidates
438 recoveries
₹42,88,918 recovered
224 bad interventions

AI Recovery Agent

408 candidates
298 recoveries
₹25,76,773 recovered
110 bad interventions

The AI was more precise.

73.0%

AI hit-rate

66.2%

baseline hit-rate

But it acted on far fewer opportunities.

408 vs. 662 candidates

Precision ↑
Volume ↓
Net revenue ↓

**The next version must recover more of the right opportunities
without sacrificing the safety improvements.**

That is the engineering lesson from the experiment.

A failed payment is not necessarily lost revenue.

AI reasons. Policy decides. Execution is controlled.

Repechage

Agentic Payment Recovery for Failed Razorpay Checkouts

Thank you.